Georgia Tech

VIRTUAL MEMORY
THE BASICS

ALEXEY TUMANOV

School of Computer Science
cc.gatech.edu/~atumanov
atumanov@gatech.edu

CREATING THE NEXT®

Administrativia

- **Lab 1** due **THIS Friday (2/06)**
  - I strongly encourage you to submit before **Tuesday (today)**.

Lab 1	JAN 23, 2026 12:00 PM	FEB 6, 2026 11:59 PM	25
Lab 1	JAN 23, 2026 12:00 PM	FEB 6, 2026 11:59 PM Late Due Date: FEB 9, 2026 11:59 PM	86

- **Lab 2** : released on **Fri 2/06**
  - Lab 2 is **much** more involved than lab 1
  - Lab 1 is NOT a proxy for Lab i complexity (esp. Lab2)
  - Use all 3 weeks to do it!
  - Lab2 due : Feb 27

ADMINISTRATION

- Lab Partner signup:
  - Go to Canvas->People->Lab Group
  - Self-assign yourself and your partner to a group
  - If working solo → assign yourself to a group anyway (unassigned: 90)
  - Lab 2 release is blocked on completing this
  - Due WED Feb 4 midnight [canvas announcement]
- Lab 2 :
  - Lab 2 is all about Virtual Memory
  - Primary goal: minimize wasted memory and unneeded copying
  - Exploration of two powerful techniques:
    - Copy on write forking
    - Lazy zero-page allocation

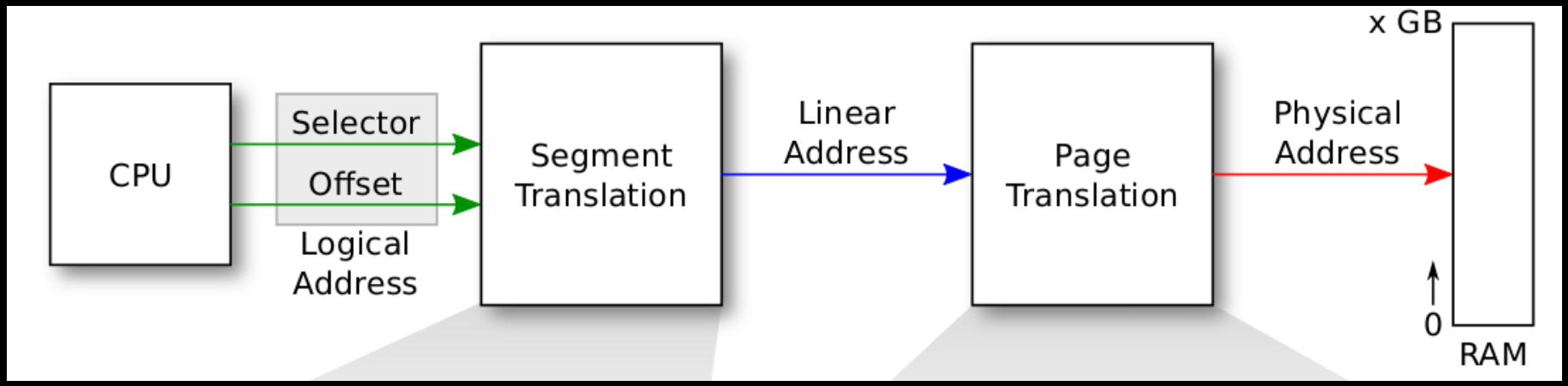
Georgia Tech

Why do we want virtual memory?

- Isolation
- Resource multiplexing
  - Makes 1 phys. addr. space look like many virt. addr. spaces
- Masks (and overcomes) resource limitations
  - Can allocate more virtual memory than we have RAM
  - Swapping
- Protection

Georgia Tech

Big picture: x86 address translation



- Segmentation vs. paging?
- Which of the above lines is the virtual address?

SEGMENTATION VS. PAGING

Georgia Tech

Segmentation

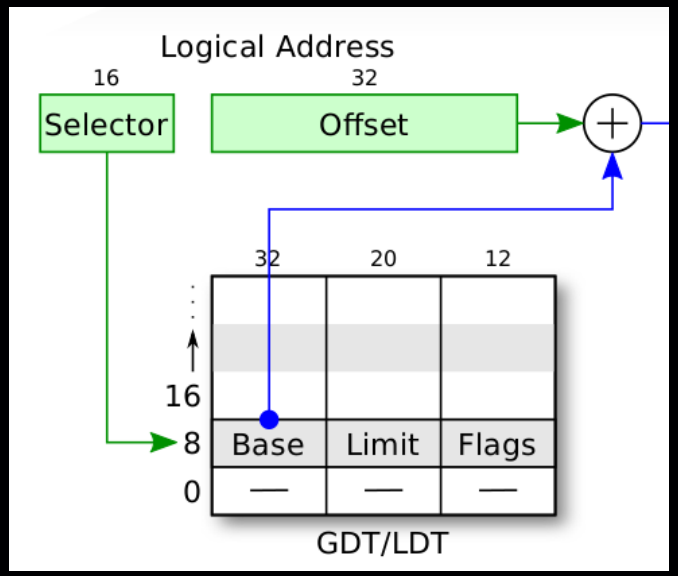
- Avoids v2p translation overhead

Paging

- Avoids fragmentation
- Allows larger virtual address spaces
- Better physical memory multiplexing

Paging: better *isolation* AND *protection* mechanism

GDT / LDT



Segmentation relies on:

- Global Descriptor Table (GDT)
  - ➢ List of memory segments which apply to all processes
- Local Descriptor Table
  - ➢ List of memory segments private to each process (used very little in modern OS)

- Segmentation registers (CS, DS, etc.) refer to GDT or LDT as appropriate
  - ➢ In this case, our GDT entries are a segment descriptor which contains:
    - a *base* (physical memory address where segment starts),
    - a *limit* (physical memory address where segment ends),
    - permissions,
    - and DPL (the segment's *Default Privilege Level*)

SEGMENTATION IN XV6 BOOTLOADER

```
# Bootstrap GDT
.p2align 2                                          # force 4 byte
alignment
gdt:
  SEG_NULLASM                                       # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)             # code seg
  SEG_ASM(STA_W, 0x0, 0xffffffff)                   # data seg
```

base

limit/range

Q: How do we specify to use code/data segments?

- We specify code/data segments using *segmentation selector registers*
  - E**SP** (***S**tack **P**ointer* register) uses **SS** (***S**tack **S**egment* register)
  - E**IP** (**I**nstruction **P**ointer register) uses **CS** (**C**ode **S**egment register)
    - EIP/CS is typically set via *jmp* instructions or based on definitions in Interrupt Descriptor Table (IDT) when interrupts occur
  - Other operations use **DS** (***D**ata **S**egment* register)

SEGMENTATION IN XV6

- When we boot xv6, we set up segments for:
  - Kernel Code (SEG_KCODE)
  - Kernel Data (SEG_KDATA)
  - User Code (SEG_UCODE)
  - User Data (SEG_UDATA)

limit/range

permissions

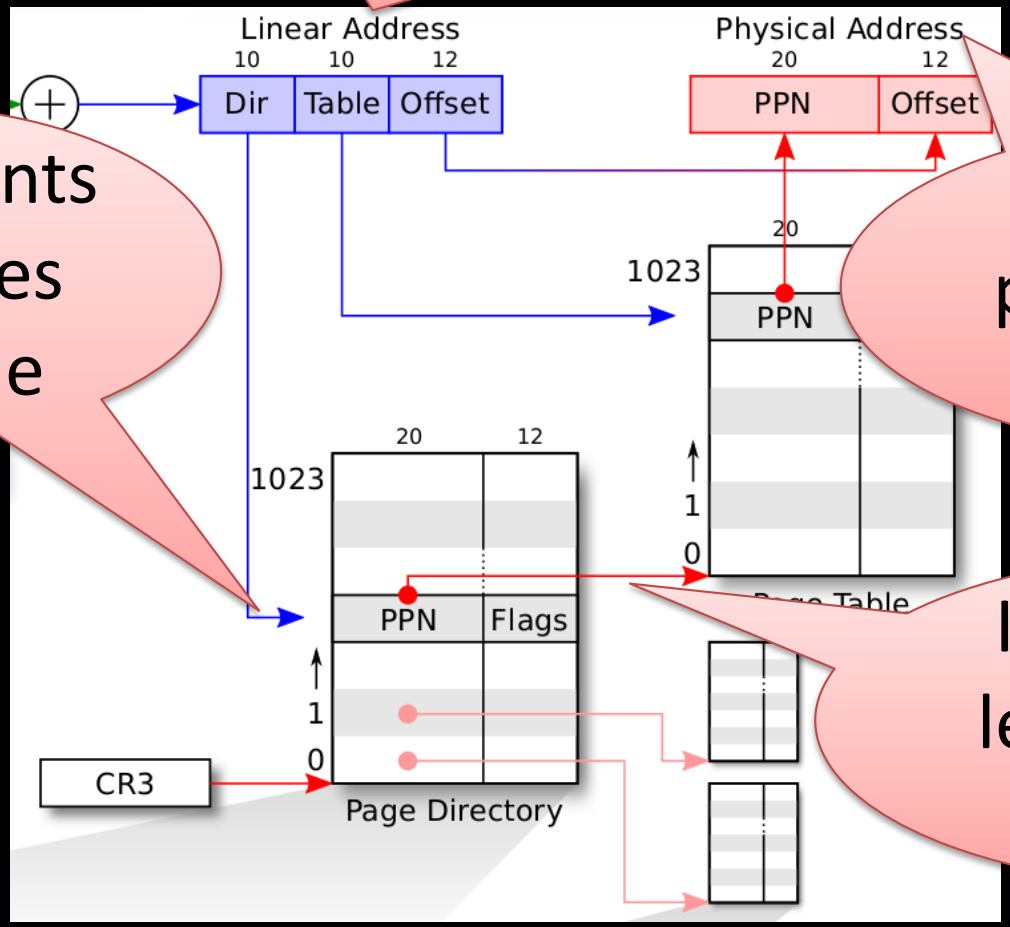
base

```
void seginit(void) {
  struct cpu *c = &cpus[cpunum()];
  c=>gdt[SEG_KCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_KERN);
  c=>gdt[SEG_KDATA] = SEG(STA_W      , 0, 0xffffffff, DPL_KERN);
  c=>gdt[SEG_UCODE] = SEG(STA_X|STA_R, 0, 0xffffffff, DPL_USER);
  c=>gdt[SEG_UDATA] = SEG(STA_W      , 0, 0xffffffff, DPL_USER);
  lgdt(c=>gdt, sizeof(c=>gdt));
  ...
}
```

Georgia Tech

Address broken down into bit segments

We've a[illegible] basics

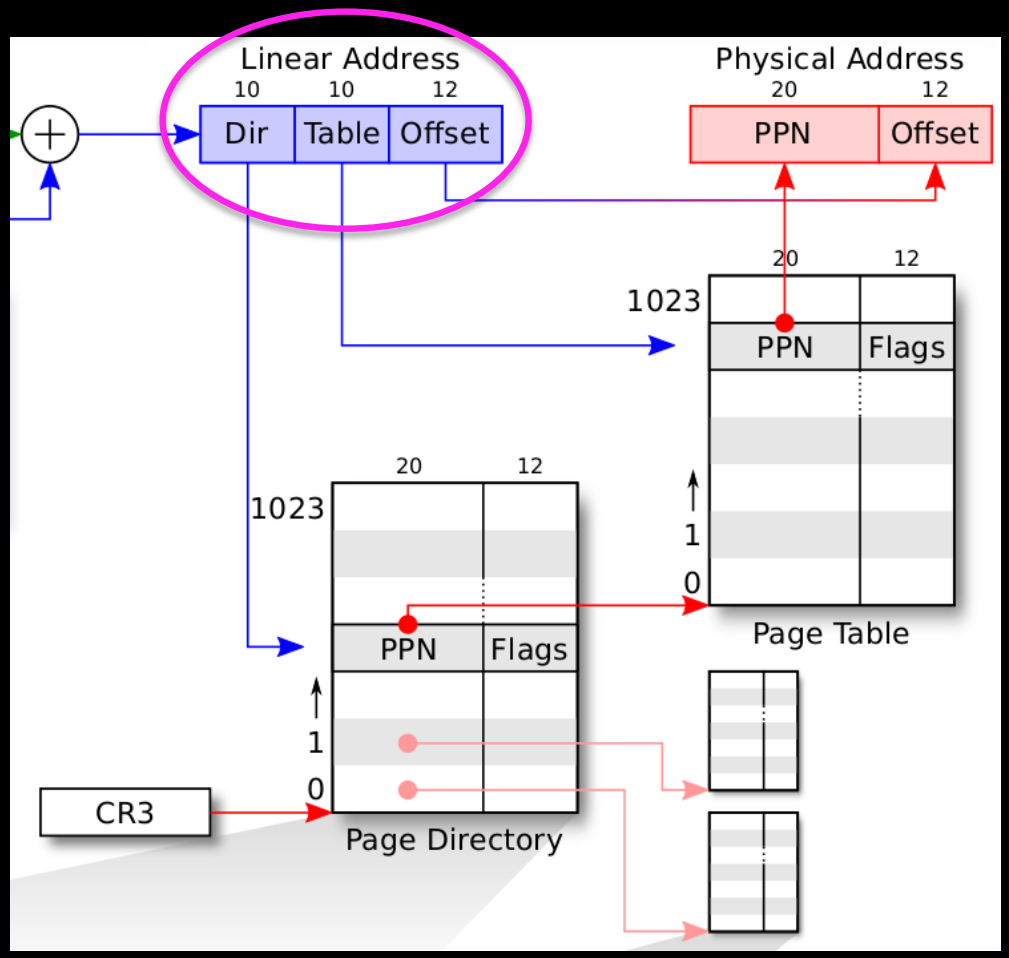


Bit segments are indices into table

Final level points to phys. addr.

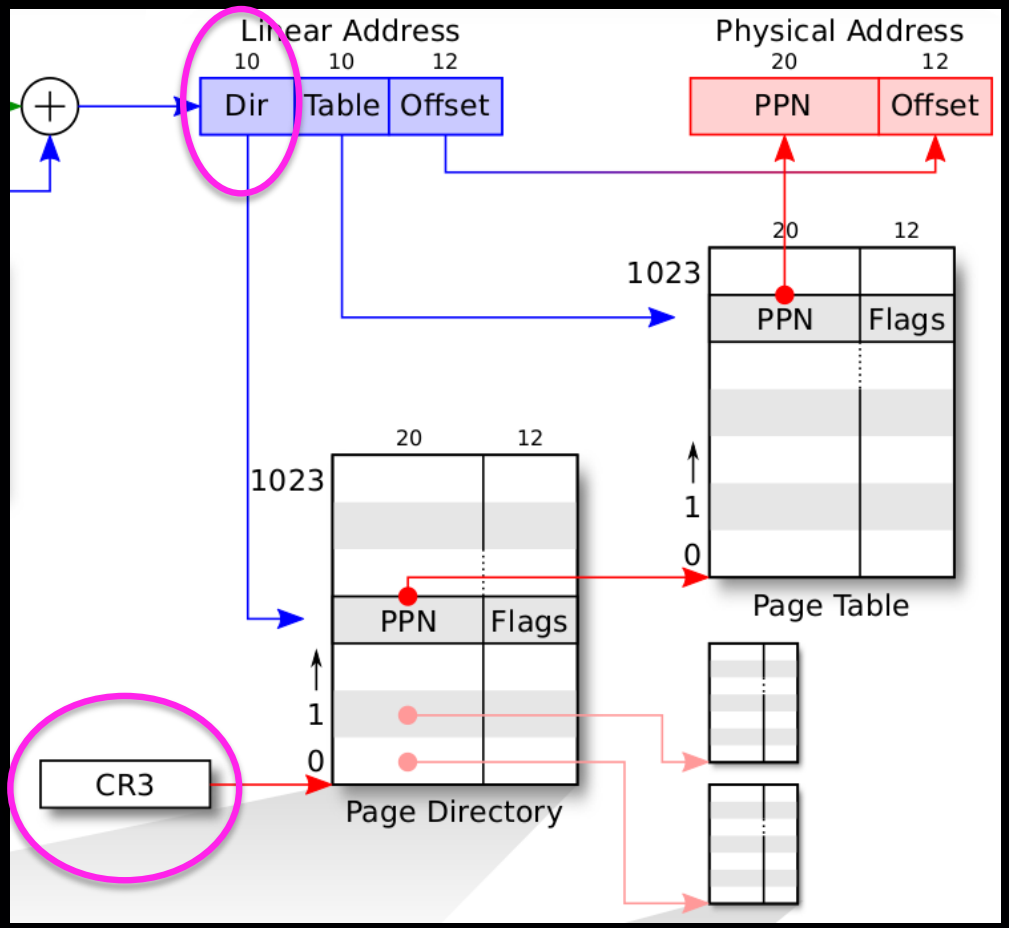
Intermediate levels point to other tables

VIRTUAL MEMORY BASICS



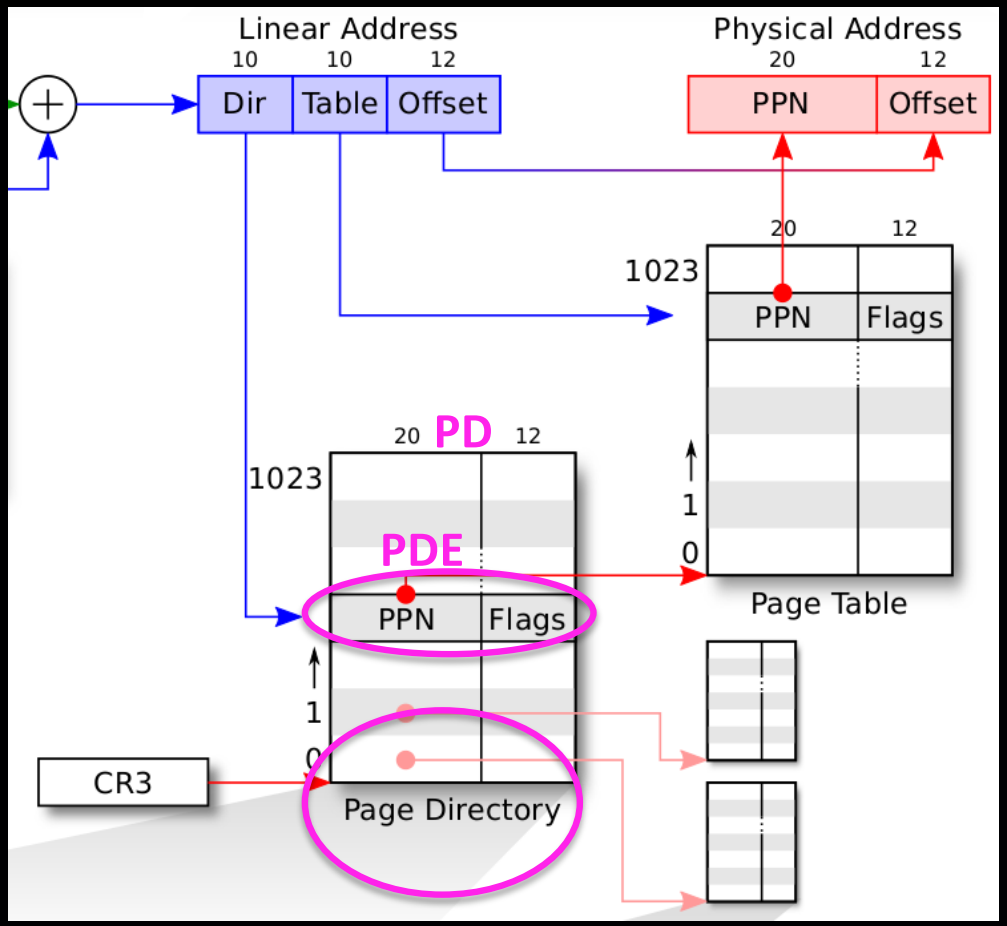
- In Virtual Memory systems, we refer to memory using *Linear* (aka *Virtual*) *Addresses*
- Each virtual address has three components:
  1. The 10 most-significant bits ("Dir") correspond to an ***index*** into the *Page Directory*
  2. The next 10 bits ("Table") correspond to an ***index*** into the *Page Table*
  3. The next 12 bits ("Offset") correspond to an ***offset*** into the actual physical address associated w/ the linear address

VIRTUAL MEMORY BASICS



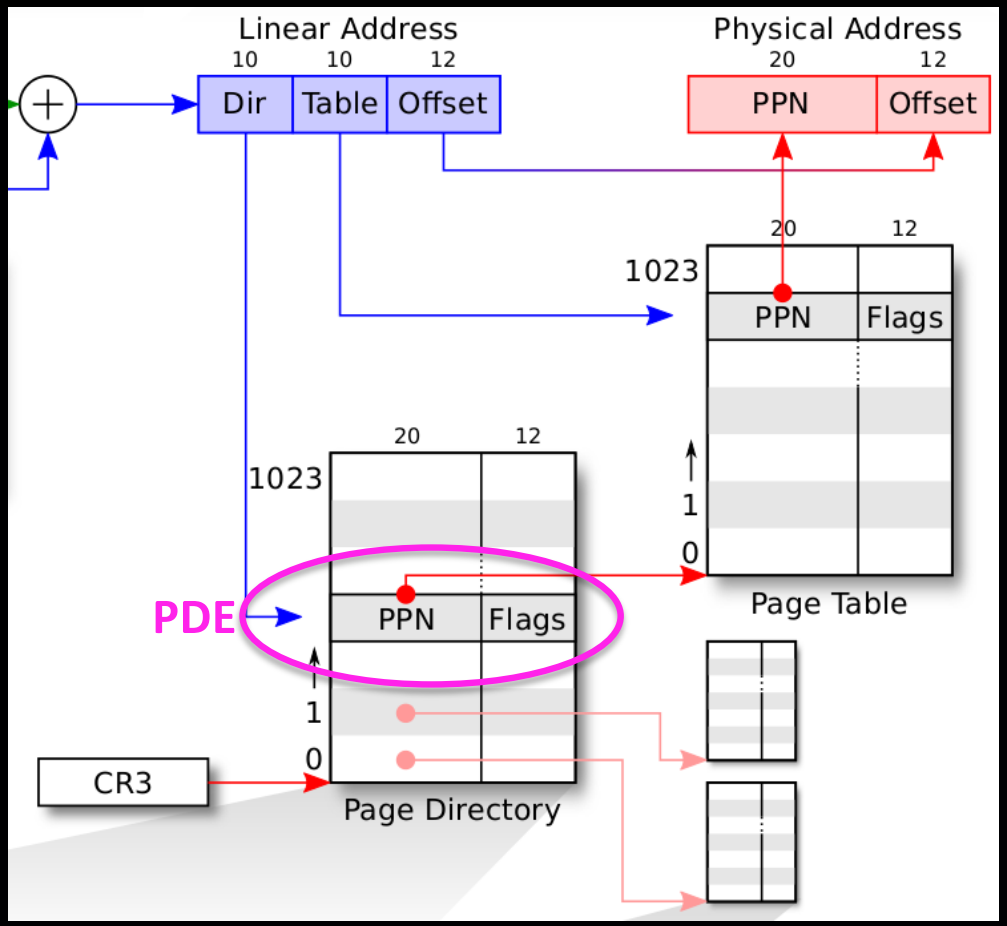
- Translation starts by loading the address of the *Page Directory* (**PD**) into **CR3** (**C**ontrol **R**egister **3**)
  - The value in CR3 points to a *physical frame* of memory which holds the *Page Directory*
  - We ***index*** into that physical frame using the 10 most-significant bits of the Linear Address

VIRTUAL MEMORY BASICS



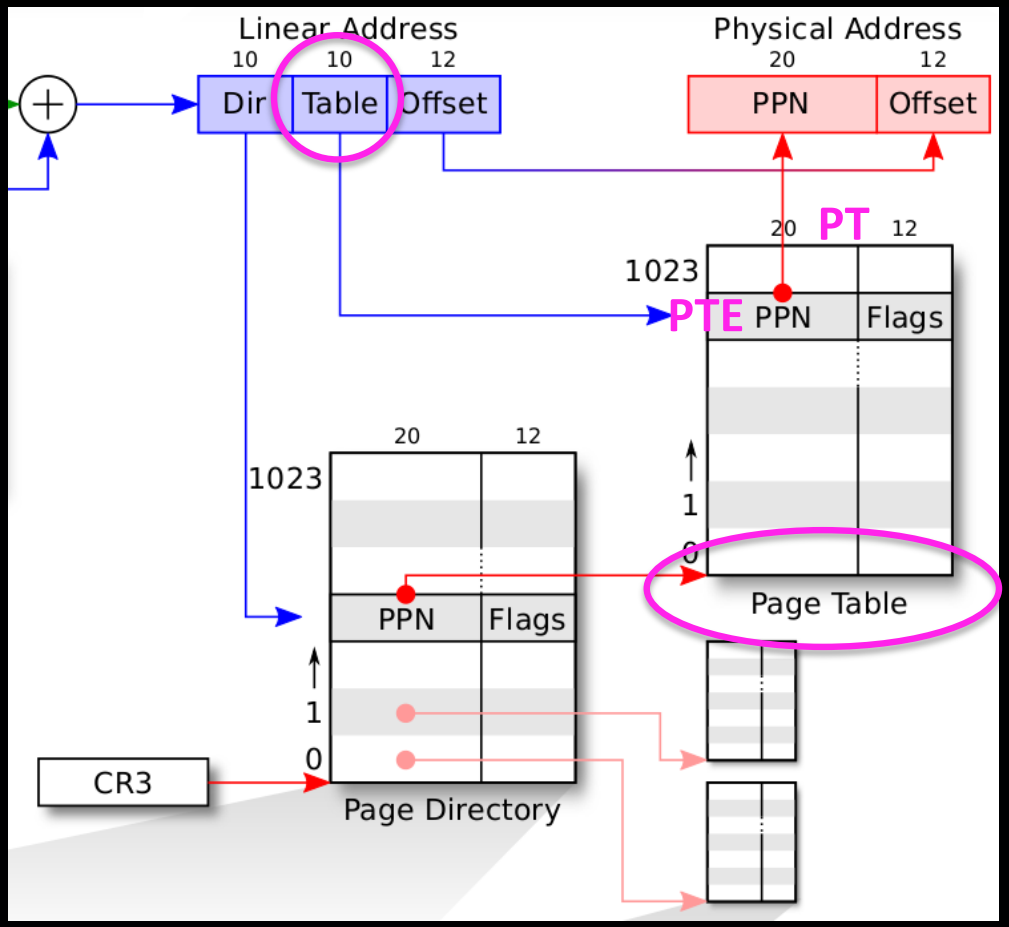
- The *Page Directory* (**PD**) can store 1024 entries
  - Why? The index value ("Dir") is a 10-bit index (2^10 = 1024)
- Each entry in the *Page Directory* (**PD**) is 32-bits
  - Each entry is called a *Page Directory Entry (***PDE***)*
- The total size of the Page Directory (PD) is 4096 bytes (4KB). Why?
  - 1024 entries * (4 bytes/entry) = 4096
- Why use 4KB for PD?
  - A single *frame* of physical memory is 4KB in size. PD efficiently uses this entire space.

VIRTUAL MEMORY BASICS



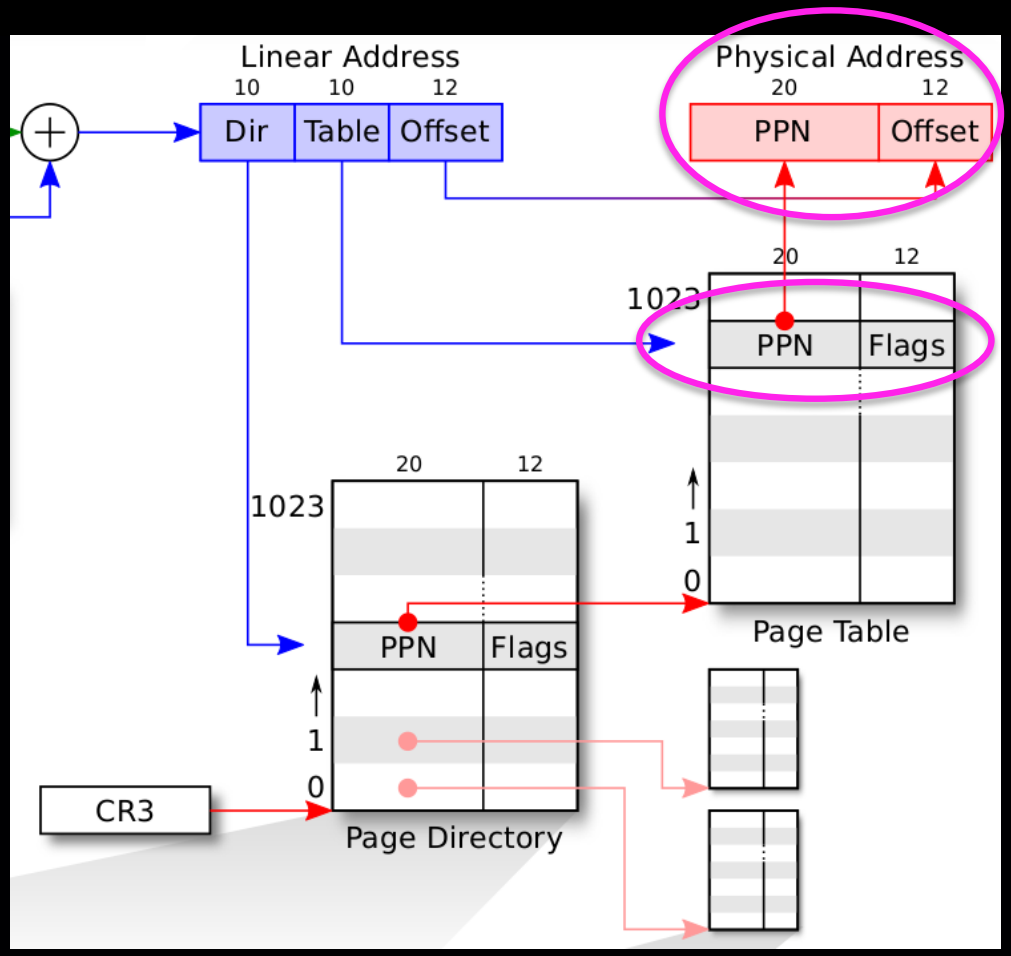
- Each *Page Directory Entry* (**PDE**) contains two parts:
  1. A *Physical Page Number* (**PPN**)
  2. A bunch of *flags* that control permissions
- A **PPN** is 20-bits in size and corresponds to the physical address of a *Page Table* (**PT**)
  - In other words, each PDE is essentially a pointer into physical memory where the next level page table is located

VIRTUAL MEMORY BASICS



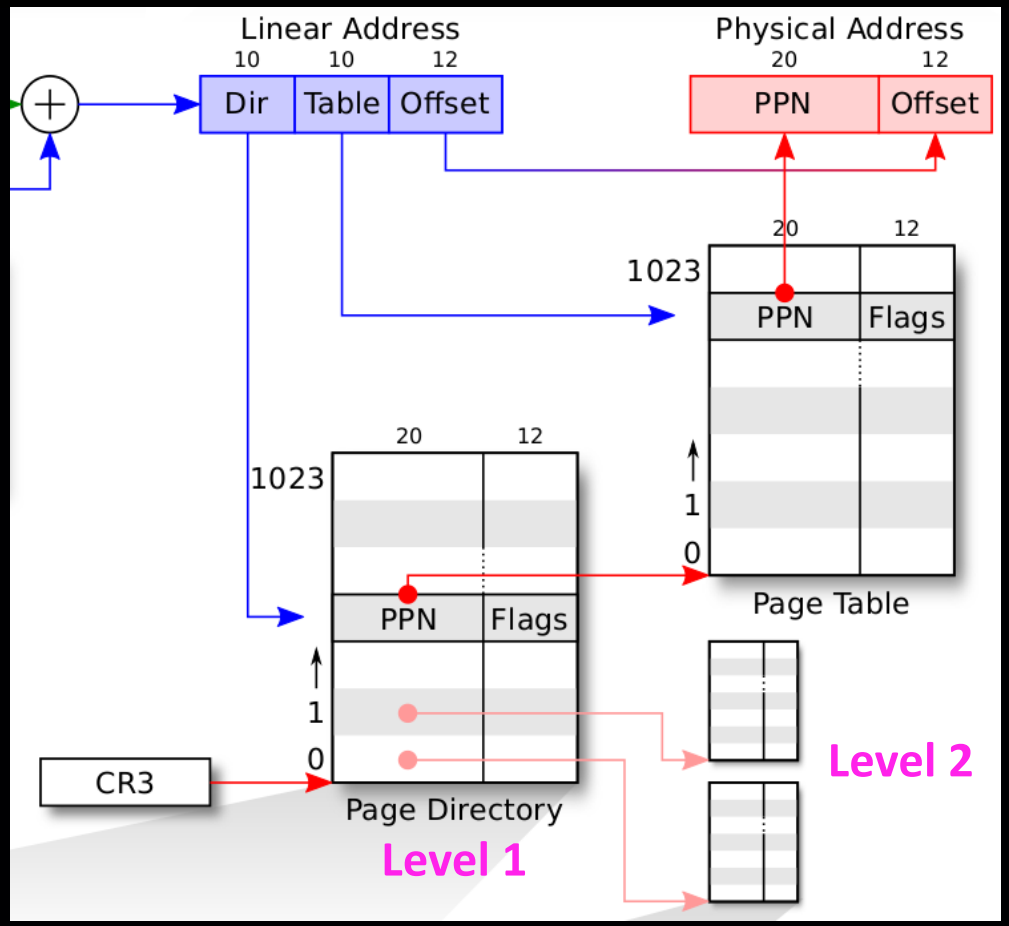
- Each virtual address space has its own set of *Page Tables* (**PT**) mapping phys.addr
  - A Page Table fits into a single frame of physical memory
- A Page Table (**PT**) consists of up to 1024 *Page Table Entries* (**PTE**s)
  - We use the next 10 most-significant bits of the Linear Address ("Table") to ***index*** into the Page Table and locate the desired Page Table Entry (PTE)

VIRTUAL MEMORY BASICS



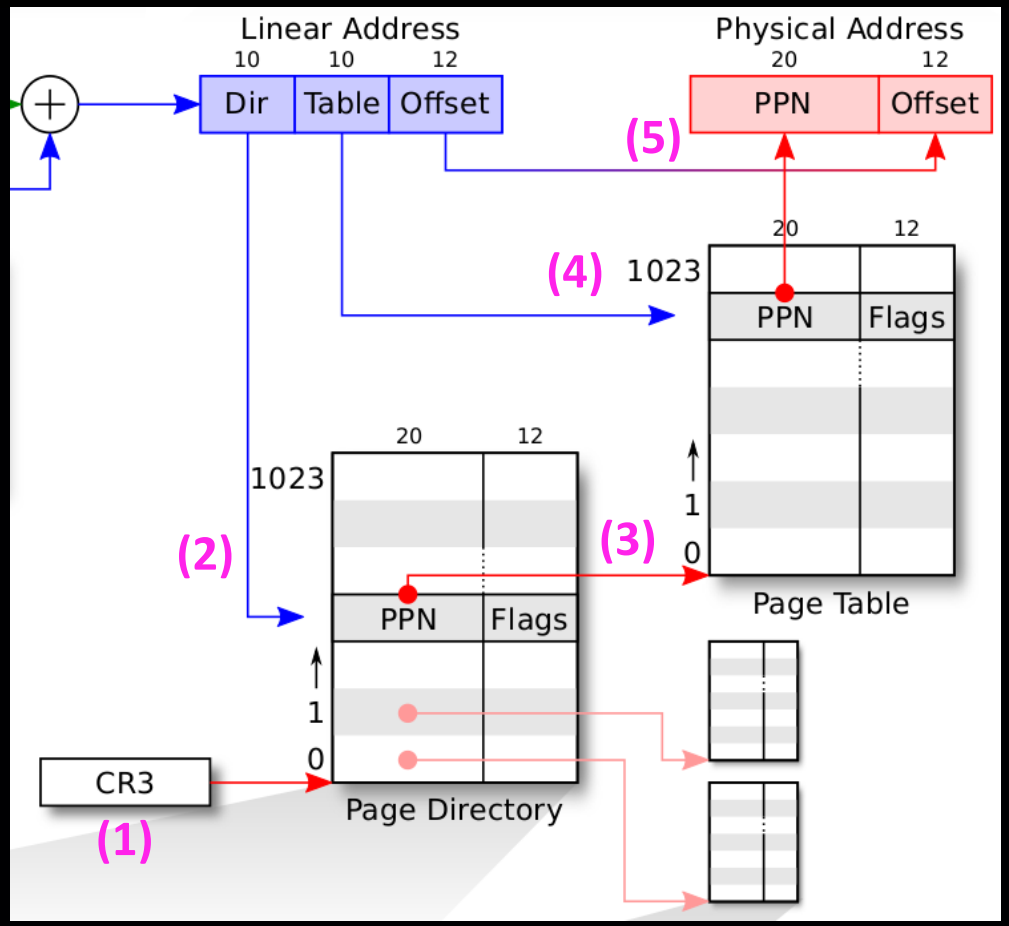
- Each *Page Table Entry* (**PTE**) points to a physical frame of memory
  - The 20 most-significant bits of the PTE corresponds with the *Physical Page Number* (**PPN**) of this frame
  - The other 12 bits of the PTE hold flags that specify what we can do with that memory
- The last 12 bits of a Linear Address correspond to an ***offset*** into the physical memory frame that is used for the page

VIRTUAL MEMORY BASICS



- Using a *Page Directory* (**PD**) and *Page Tables* (**PT**s) gives us a *Two-Level Hierarchy* for managing virtual memory
  - The Page Directory is at the first level and references Page Tables
  - Each Page Table is at the second level and references physical memory frames

VIRTUAL MEMORY BASICS

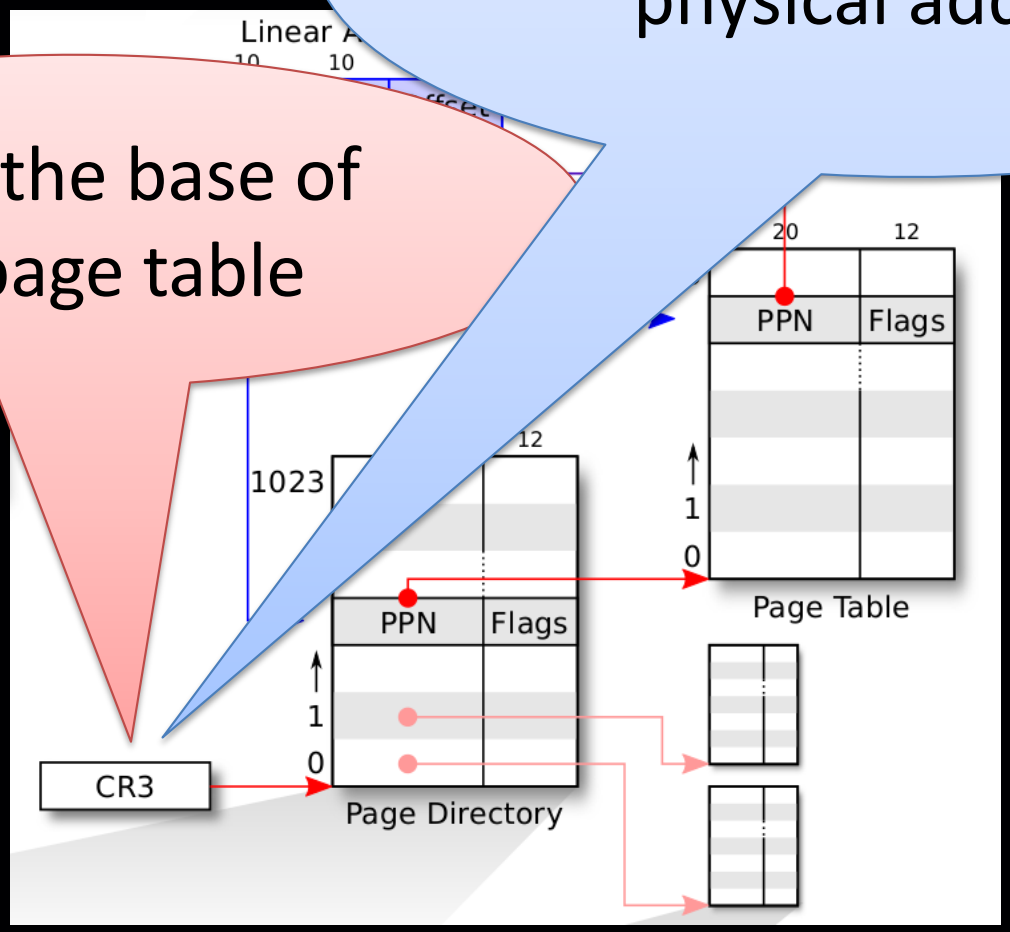


- Virtual Address Translation Recap
  1. Load Page Directory (PD) address into CR3
  2. Use 10 most-significant bits of Linear Address ("Dir") to ***index*** into PD and retrieve a Page Directory Entry (PDE)
  3. Use PDE to locate the Page Table in physical memory
  4. Use next 10 most-significant bits of Linear Address ("Table") to ***index*** into PT and locate Physical Page Number (PPN)
  5. Use last 12 bits of Linear Address ("Offset") to ***offset*** into the PPN

Page Translatio

Is CR3 a virtual or physical address?

CR3 is the base of the page table



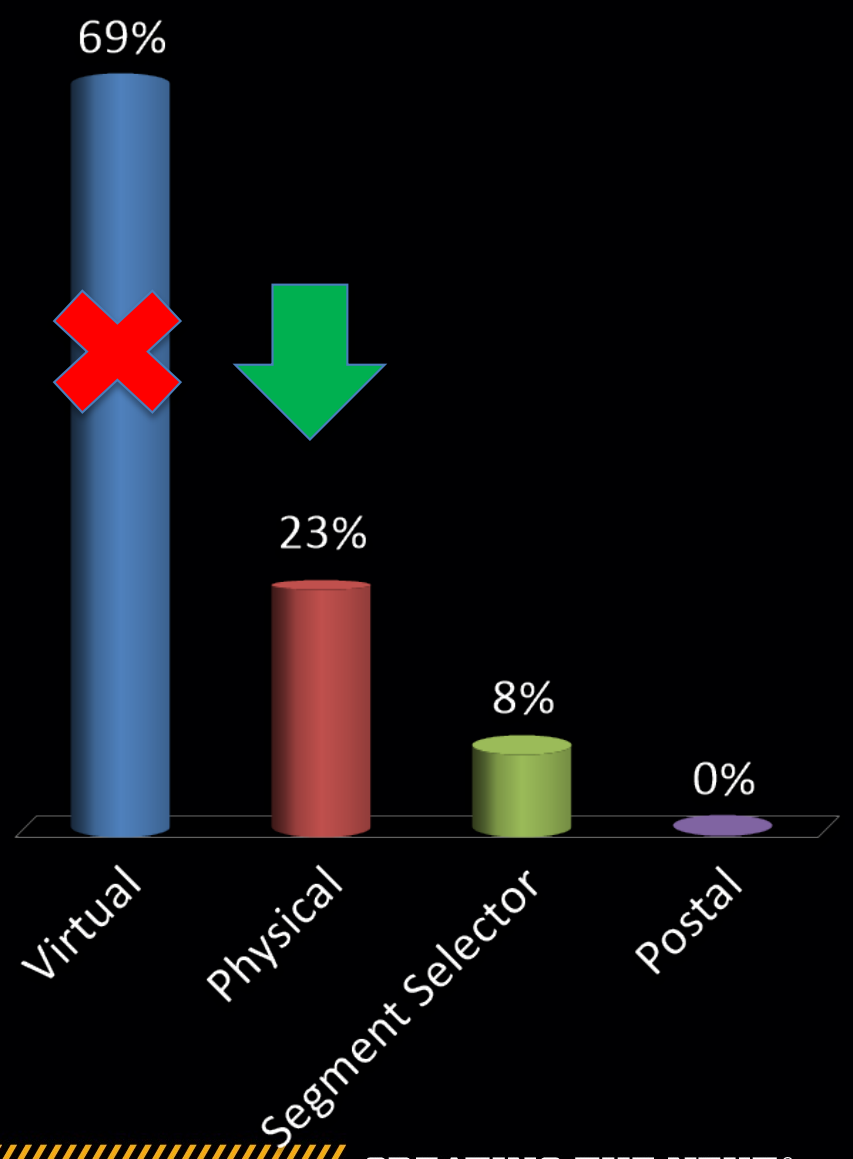
WHAT TYPE OF ADDRESS IS STORED CR3?

A. Virtual

B. Physical

C. Segment Selector

D. Postal

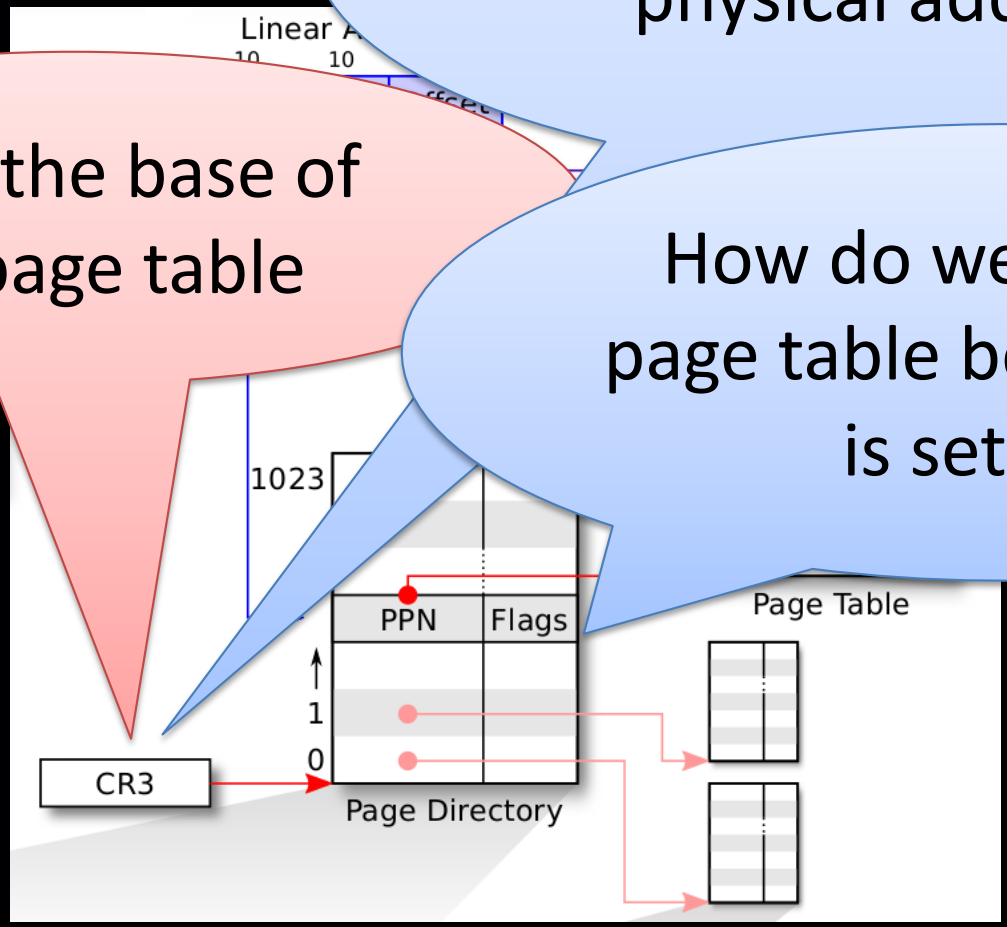


Page Translatio

Is CR3 a virtual or physical address?

CR3 is the base of the page table

How do we make the page table before paging is setup??



Georgia Tech

What must the paging system handle?

- Mapping virtual to physical addresses
- Physical page allocation!
  - Especially challenging in bootstrapping!

But, before we talk about bootstrapping…

How big is the single level page?

x86 single-level page

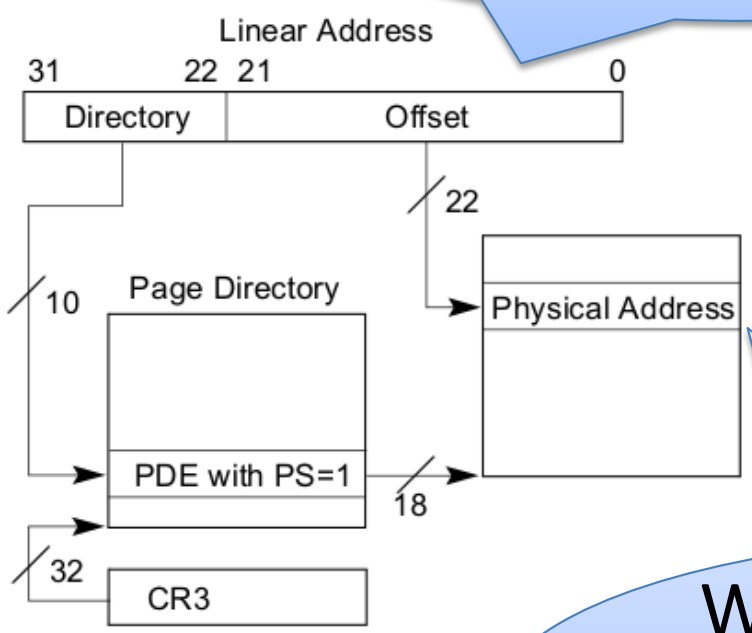


Figure 4-3. Linear-Address Translation to a 4-

Why would we want larger or smaller pages?

- Q: page size?
- Q: trade-off (better/worse)?

32bit: 4KB, 4MB **64bit**: 4KB, 2MB, 1GB

Georgia Tech

Bootup page-table in xv6 – a single 4MB page!

```
// main.c
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Q?
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Q?
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

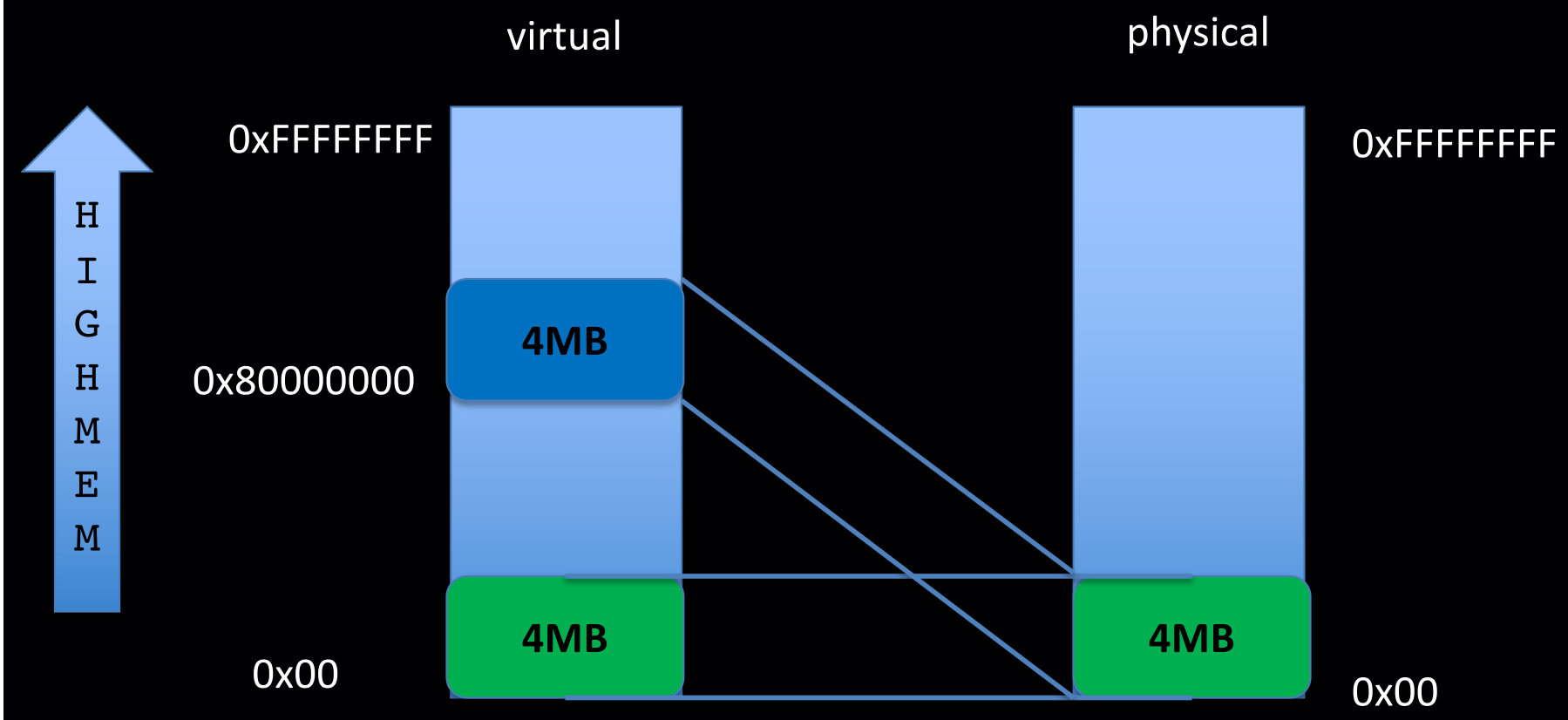
VFN

kernel/src/main.c

PFN

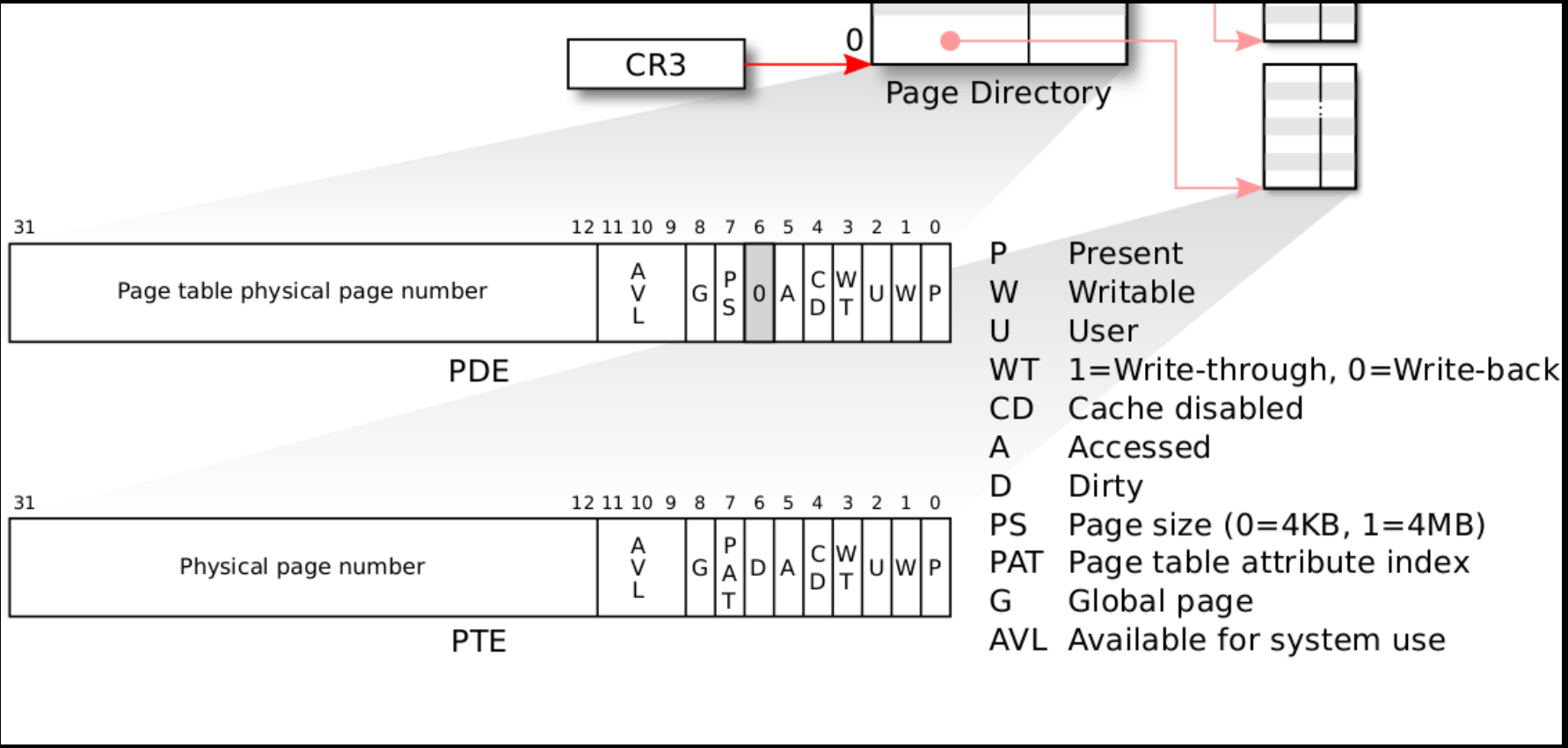
```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

ENTRYPGDIR


virtual
physical
0xFFFFFFFF
0xFFFFFFFF
HIGHMEM
4MB
0x80000000
4MB
4MB
0x00
0x00

Georgia Tech

(PTE/PDE)



Georgia Tech

PTE/PDE: how to interpret?

```
// main.c
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Q?
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Q?
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

OS DESIGN: MULTILEVEL PAGING

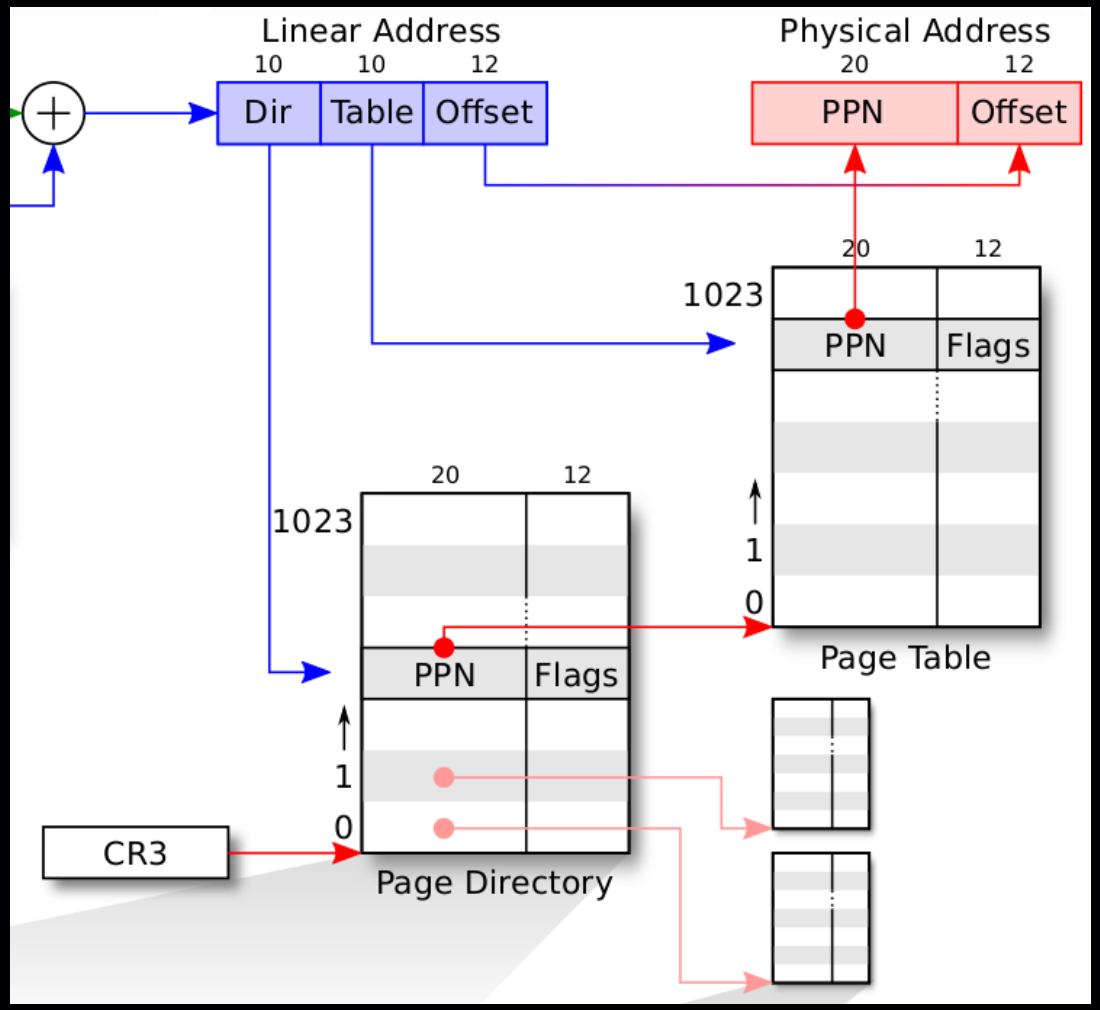
Multi-level paging

- Why not just 4K single-level paging?
  - i.e., 2^20 x size(pte) = ?? MB
- Why problematic?
  - size
  - performance
  - fragmentation

Why advantageous:

Page Translation (two-level)

- **Given a virtual address, how many memory lookups to *translate* it to a physical address (aka page walk)?**



HOW MANY MEMORY OPERATIONS IN A 2-LEVEL PAGE TABLE WALK?

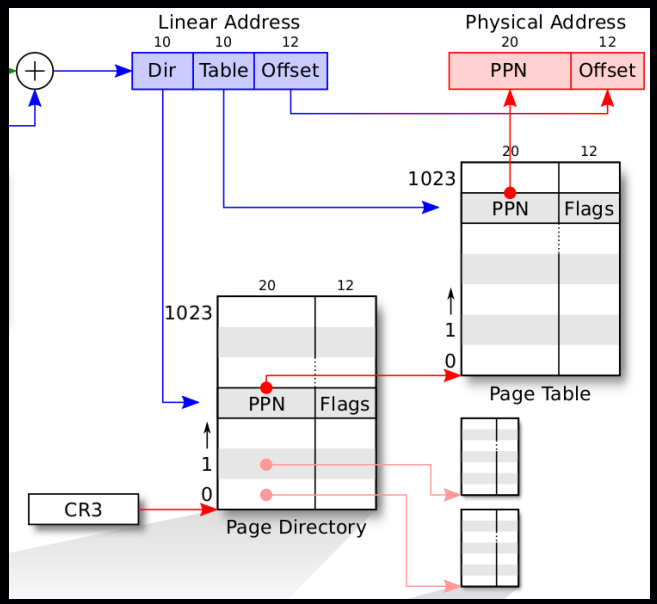
A. 0

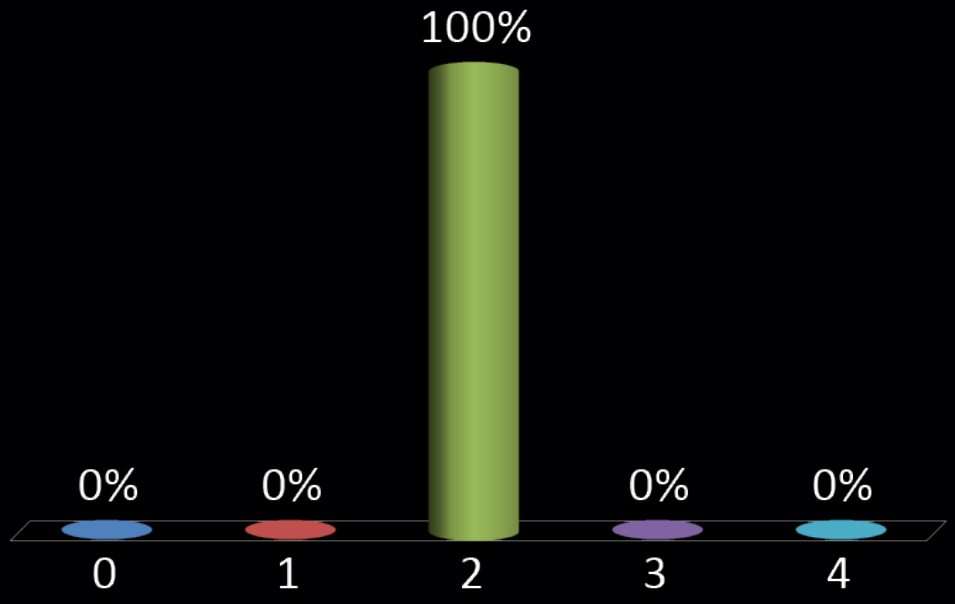
B. 1

C. 2

D. 3

E. 4





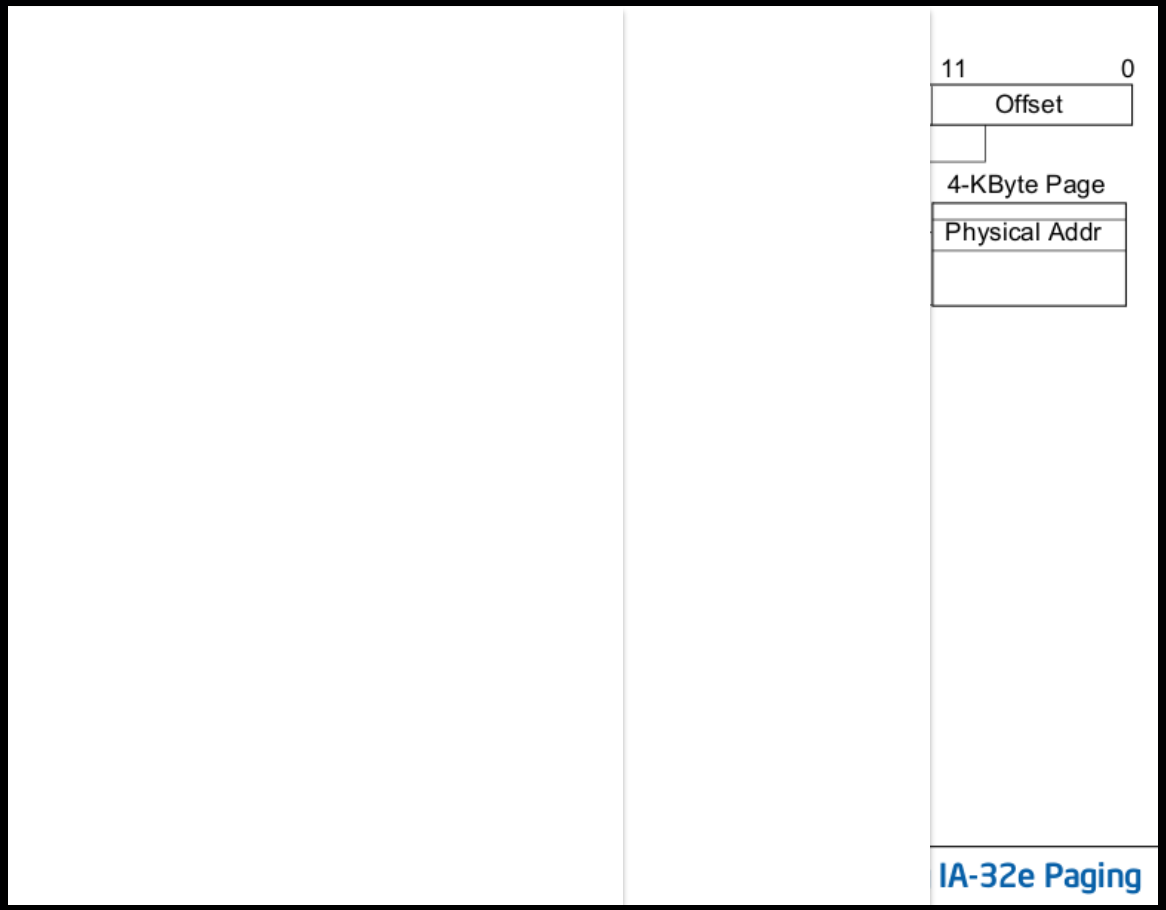
MULTILEVEL PAGING DESIGN

- *Two* level paging (10 10 12)
- *One* level paging (10 22)
  - Use case: small process with minimal mem usage
  - Resource waste (~8-12MB min most processes)
- *One* level paging (20 12)
  - Use case: small processes ? Many processes?
- *Overheads* to support the paging mechanism:
  - Memory overhead (sum of all PTPs)
  - Latency overhead (page table walk)

MULTILEVEL PAGING DESIGN: X86_64

Four-level paging in x86-64

Same idea, more levels of indirection.



MULTILEVEL PAGING DESIGN: X86_64

Four-level paging in x86-64

Same idea, more levels of indirection.

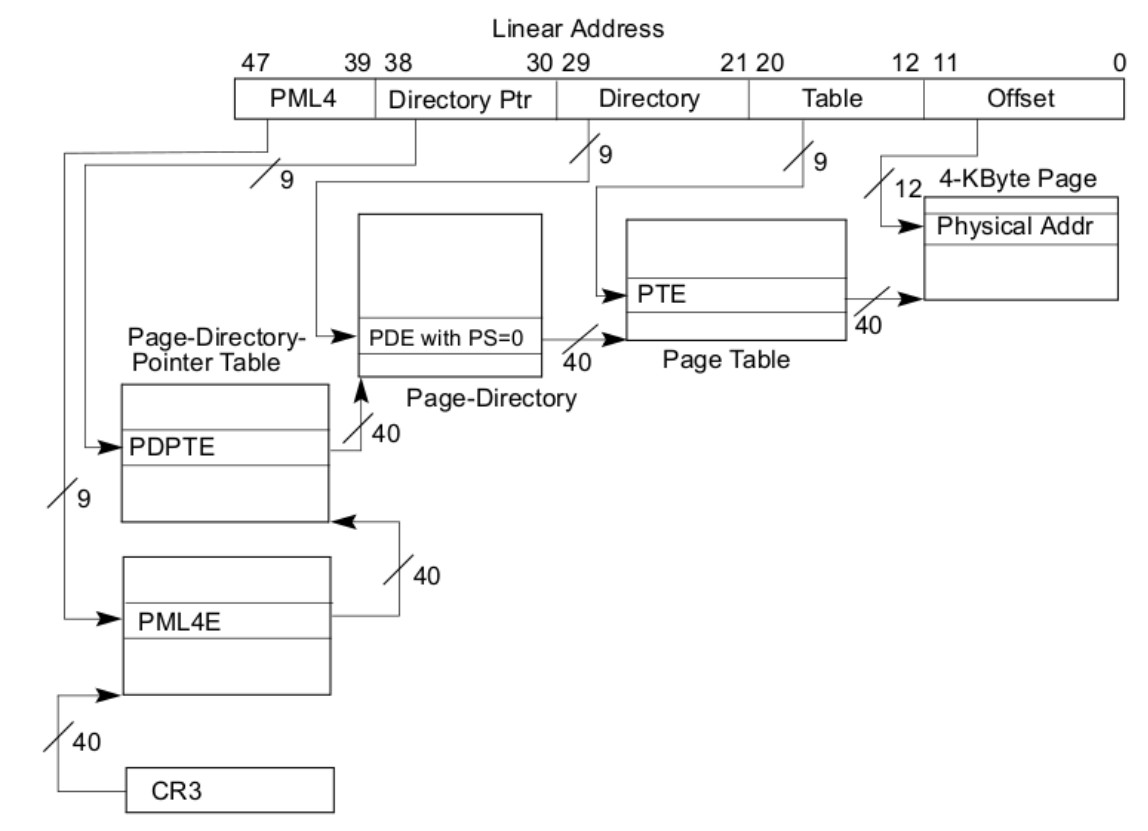


Figure 4-8. Linear-Address Translation to a 4-KByte Page using IA-32e Paging

MULTILEVEL PAGING DESIGN

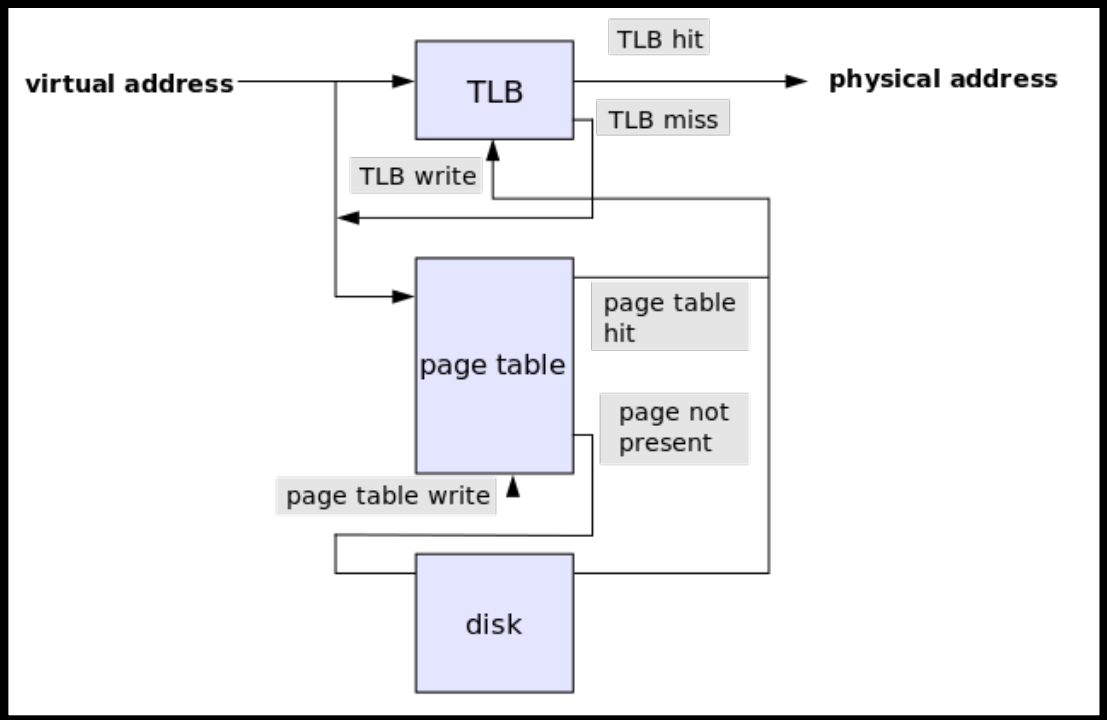
- *Overheads* to support the paging mechanism:
  - Memory overhead (sum of all PTPs)
  - Latency overhead (page table walk)

Memory Management Unit (MMU)

Hardware support: VA -> PA translation

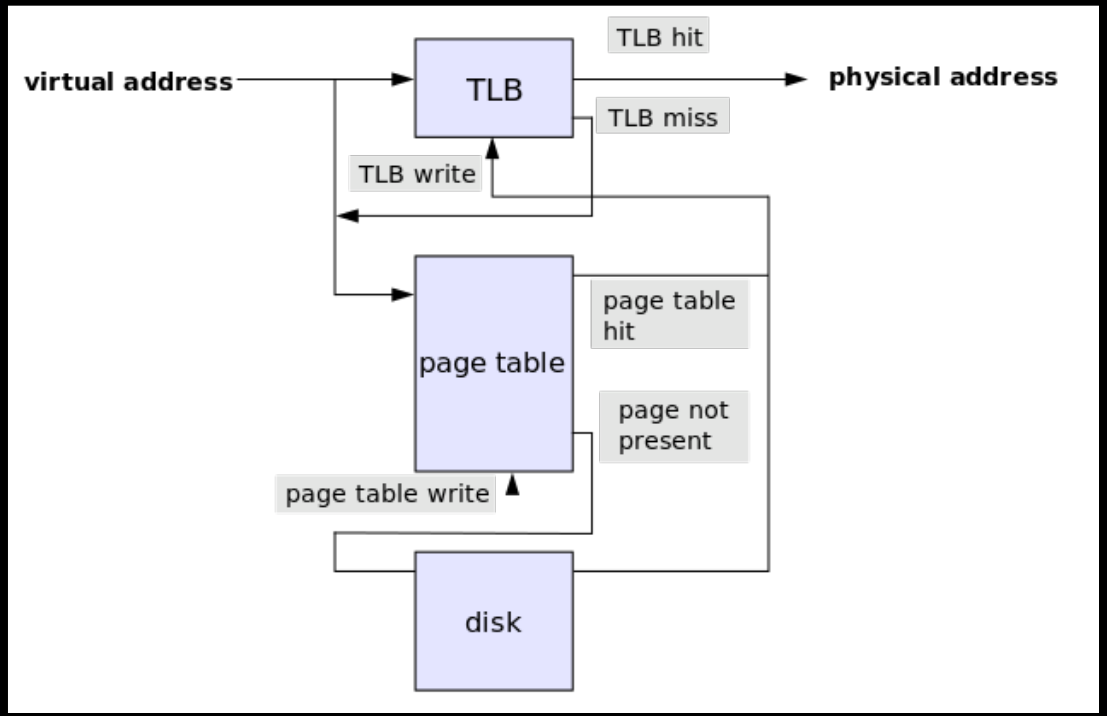
Cache: TLB (translation lookaside buffer)

- Caching mappings: virtual -> physical addresses
- Optimization: iTLB, dTLB



MEMORY MANAGEMENT UNIT (MMU)

- Virtual Address (VA) to Physical Address (PA) translation is supported in hardware via the MMU
- There is a special hardware cache for VA->PA translations called the *Translation Lookaside Buffer* (**TLB**)
  - Optimizations to TLB also exist
    - e.g., iTLB only caches *instructions*
    - e.g., dTLB only caches *data*
  - Note: Changing CR3 (e.g., when we switch processes) causes a *TLB Flush* which wipes the cache. This is part of the reason switching processes is so expensive.



Georgia Tech

Code: Bootstrapping paging in xv6

Recall:

bootasm.S (real to protected) -> bootmain (protected, load MBR) -> entry -> ... main

- kinit1() in kalloc.c (physical page allocator)
- kvmalloc() in main.c (kernel page table)

Georgia Tech

Virtual address space in xv6

```
          +-=> +-------------------+  <= 0xFFFFFFFF
           |   |                   |
           |   |     free memory   |
           |   +-------------------+
kernel     |   | kernel text/data  |  4MB
space      |   +-------------------+  <= 0x80100000
(CPL=0)    |   |       BIOS        |
          +-=> +-------------------+  <= 0x80000000
           |   |       heap        |
           |   +-------------------+
user       |   |       stack       |
space      |   +-------------------+
(CPL=3)    |   |  user text/data   |
          +-=> +-------------------+  <= 0x00000000
```

Georgia Tech

The first address space in xv6

```
                          +------------------+  <= 0xFFFFFFFF
                          |                  |
                          |    free memory   |
                         ++------------------+
                        / | kernel text/data |  (kernel)
                       /  +------------------+  <= 0x80100000
                      +   |       BIOS       |
   physical mem      /   ++------------------+  <= 0x80000000
                    /   / |       heap       |  (KERNBASE)
+------------------+   | +------------------+
| kernel text/data |   + |       stack      |
+------------------+  /  +------------------+
|       BIOS       |/    |  user text/data  |  (initcode)
+------------------+     +------------------+  <= 0x00000000
```

Georgia Tech

Physical Page Management in xv6

Boot Allo

```
__attribute__((__aligned__(PGSIZE)))
pde_t entrypgdir[NPDENTRIES] = {
  // Map VA's [0, 4MB) to PA's [0, 4MB)
  [0] = (0) | PTE_P | PTE_W | PTE_PS,
  // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)
  [KERNBASE>>PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,
};
```

```
# Enteri
 .globl
 entry:
   # Tur
   movl
   orl
   movl     %eax, %cr4
   # Set page directory
   movl     $(V2P_WO(entrypgdir)), %eax
   movl     %eax, %cr3
   # Turn on paging.
   movl     %cr0, %eax
   orl      $(CR0_PG|CR0_WP), %eax
   movl     %eax, %cr0
```

xv6 uses a single 4M superpage with entrypgdir as the directory to bootstrap the kernel

Georgia Tech

Physical Page Management in xv6

```
// Initialization happens in two phases.
// 1. main() calls kinit1() while still using entrypgdir to place just
// the pages mapped by entrypgdir on free list.
// 2. main() calls kinit2() with the rest of the physical pages
// after installing a full page table that maps them on all cores.
void
kinit1(void *vstart, void *vend)
{
  initlock(&kmem.lock, "kmem");
  kmem.use_lock = 0;
  freerange(vstart, vend);
}

void
freerange(void *vstart, void *vend)
{
  char *p;
  p = (char*)PGROUNDUP((uint)vstart);
  for(; p + PGSIZE <= (char*)vend; p += PGSIZE)
    kfree(p);
}
```

Georgia Tech

Physical Page Management in xv6

- xv6 embeds the page info in the free pages (no page sharing)
- kfree in kalloc.c adds to the linked list of free pages.

```
// Free the page of physical memory pointed at by v, which normally should
// have been returned by a call to kalloc().  (The exception is when
// initializing the allocator; see kinit above.)
void
kfree(char *v)
{
  struct run *r;
  if((uint)v % PGSIZE || v < end || V2P(v) >= PHYSTOP)
    panic("kfree");
  // Fill with junk to catch dangling refs.
  memset(v, 1, PGSIZE);
  if(kmem.use_lock)
    acquire(&kmem.lock);
  r = (struct run*)v;
  r->next = kmem.freelist;
  kmem.freelist = r;
  if(kmem.use_lock)
    release(&kmem.lock);
}
```

Georgia Tech

Page Table management in xv6

```
// This table defines the kernel's mappings, which are present in
 // every process's page table.
 static struct kmap {
   void *virt;
   uint phys_start;
   uint phys_end;
   int perm;
 } kmap[] = {
  { (void*)KERNBASE, 0,             EXTMEM,    PTE_W}, // I/O space
  { (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},     // kern text+rodata
  { (void*)data,     V2P(data),     PHYSTOP,   PTE_W}, // kern data+memory
  { (void*)DEVSPACE, DEVSPACE,      0,         PTE_W}, // more devices
 };
```

Georgia Tech

Page Table Management in xv6

```
// Set up kernel part of a page table.
pde_t*
setupkvm(void)
{
  pde_t *pgdir;
  struct kmap *k;
  if((pgdir = (pde_t*)kalloc()) == 0)
    return 0;
  memset(pgdir, 0, PGSIZE);
  if (P2V(PHYSTOP) > (void*)DEVSPACE)
    panic("PHYSTOP too high");
  for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
    if(mappages(pgdir, k->virt, k->phys_end - k->phys_start,
                (uint)k->phys_start, k->perm) < 0) {
      freevm(pgdir);
      return 0;
    }
  return pgdir;
}
```

Page Table Management in xv6

- xv6 has a function that maps pages (sets up Page Table Entries)
- walkpgdir finds PTE from a virtual address (what the x86 hardware does)

```
static int
mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)
{
  char *a, *last;
  pte_t *pte;
  a = (char*)PGROUNDDOWN((uint)va);
  last = (char*)PGROUNDDOWN(((uint)va) + size - 1);
  for(;;){
    if((pte = walkpgdir(pgdir, a, 1)) == 0)
      return -1;
    if(*pte & PTE_P)
      panic("remap");
    *pte = pa | perm | PTE_P;
    if(a == last)
      break;
    a += PGSIZE;
    pa += PGSIZE;
  }
  return 0;
}
```

Georgia Tech

Page Walking in xv6

```
static pte_t *
walkpgdir(pde_t *pgdir, const void *va, int alloc)
{
  pde_t *pde;
  pte_t *pgtab;
  pde = &pgdir[PDX(va)];
  if(*pde & PTE_P){
    pgtab = (pte_t*)P2V(PTE_ADDR(*pde));
  } else {
    if(!alloc || (pgtab = (pte_t*)kalloc()) == 0)
      return 0;
    // Make sure all those PTE_P bits are zero.
    memset(pgtab, 0, PGSIZE);
    // The permissions here are overly generous, but they can
    // be further restricted by the permissions in the page table
    // entries, if necessary.
    *pde = V2P(pgtab) | PTE_P | PTE_W | PTE_U;
  }
  return &pgtab[PTX(va)];
}
```

Georgia Tech

So, why is paging good?

Potential applications:

Kernel tricks (e.g., one zero-filled page)

Faster system calls (e.g., copy-on-write fork)

New features (e.g., memory-mapped files)

Georgia Tech

xv6 vs other operating systems

xv6 (currently) has no ability to have multiple virtual pages map to a single page

- Makes Copy-on-Write almost impossible
- Shared memory mapping difficult

xv6 expects the kernel to map all of physical memory (up to 2GB) including all user programs

- Any user program can map virtual pages to free physical pages
- Once the kernel and a user program (at most) have a mapping to a physical page, the physical page can have no more mappings.
- Physical Page mapping is simple and binary (free or used)

Other OS can have many user programs map virtual pages to the same physical page

- Enables copy-on-write support
- Shared libraries need only be in one place in physical memory and mapped over and over
- Means that page references must be tracked

RESEARCH PERSPECTIVE

Snowflock: rapid virtual machine cloning for cloud computing

European Conference on Computer Systems (EuroSys 2009)

- ``forked'' a VM in under a second to a networked cluster''
- On-demand paging, CoW, selective page transfer

Kaleidoscope: cloud micro-elasticity via VM state coloring

EuroSys 2011

- Efficient stateful VM replication

On-Demand-Fork: A Microsecond Fork for Memory-Intensive and Latency-Sensitive Applications, EuroSys 2021!